SIEMENS

Functions for software units 1

Functions for fail-safe unit 2

TIA Portal Openness: API for automation of engineering workflows

Manual

21.00.00.00

12/2025

NOTICE

This document was downloaded from the following site:

https://docs.industrial-operations-x.siemens.cloud

Siemens assumes no liability for the accuracy, completeness, or timeliness of the data provided.

Use is at your own risk.

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

DANGER

indicates that death or severe personal injury **will** result if proper precautions are not taken.

WARNING

indicates that death or severe personal injury **may** result if proper precautions are not taken.

CAUTION

indicates that minor personal injury can result if proper precautions are not taken.

NOTICE

indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

⚠ WARNING

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

12/2025 Subject to change

Copyright © Siemens 2026.
All rights reserved

Table of contents

1 Functions for software units 6

1.1 Accessing software unit 6
1.2 Working with software unit 9
1.3 Accessing software unit underlying objects 13
1.4 Accessing to existing relations of a unit 17
1.5 Updating software unit properties 20
1.6 Publishing software unit object 23
1.7 Adding external sources in units 27
1.8 Units as mastercopies 30
1.9 Updating existing relations & create/delete relations 33
1.10 Accessing name value type document 42
1.11 Accessing namespaces for software units 43
1.12 Accessing namespaces for program blocks 44
1.13 Accessing namespaces for UDT 46
1.14 Generating loadable file for software units 49

2 Functions for fail-safe unit 54

2.1 Generating fail-safe unit 54
2.2 Accessing the SafetyUnit 56
2.3 Creating/deleting SafetyUnit relations 59
2.4 Creating/Accessing SafetyUnit elements/objects 60
2.5 Exporting/Importing supervisions under SafetyUnit 64
2.6 Publishing blocks under the SafetyUnit 66

Functions for software units

1

1.1 Accessing software unit

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to access the units. The units are important parts of the PLC's programming. Only a selected set of PLCs are supporting units therefore units are not modelled statically in the object model so they aren’t browsable directly under the PLC software container.

The PlcUnitProvider makes units accessible and is accessible from the PlcSoftware through GetService( ). The PlcUnitProvider allows the access to PlcUnitSystemGroup via the UnitGroup navigator which contains the PlcUnitComposition with the general composition specific possibilities (iterate, Find, Create, Import etc. for PlcUnits).

The unit's representation (PlcUnit) contains the unit's own specific properties as Openness attributes.

You can access the below attribute and method for Openness unit properties:

Attribute Name	Type
Author	String
Name	String

Method Name	Return type
Export	void
Delete	void
GetAttributes	IList<object>

SetAttributes	void

On the PlcUnit, You can iterate through the following objects:

- Comment: MultilingualText
- the BlockGroup (PlcBlockSystemGroup)
  - unit.BlockGroup
- TagTableGroup (PlcTagTableSystemGroup)
  - unit.TagTableGroup
- TypeGroup (PlcTypeSystemGroup)
  - unit.TypeGroup

Program code

 Copies the following program code to clipboard.

```
private static PlcUnit AccessSoftwareUnit(Project project,
string plcName = "PLC_1", string plcUnitName = "Unit_1")
{
PlcUnit plcUint = null;
Device device = null;
DeviceItem deviceItem = null;
if (project == null)
{
Console.WriteLine("Project cannot be null");
return plcUint;
}
//Finding the plc
Console.WriteLine(String.Format("Iterate through {0}
device(s)", project.Devices.Count));
foreach (Device checkDevice in project.Devices)
{
if (device == null)
{
Console.WriteLine(String.Format("Device: \"{0}\".",
checkDevice.Name));
```

```
{
if (device == null)
{
if (checkDeviceItem.Name == plcName)
{
device = checkDevice;
deviceItem = checkDeviceItem;
}
}
}
}
}
if (device == null)
{
Console.WriteLine("Device cannot be null");
return plcUint;
}
else
{
SoftwareContainer softwareContainer =
((IEngineeringServiceProvider)deviceItem).GetService<SoftwareContainer>
();
PlcSoftware software = softwareContainer.Software as PlcSoftware;
//Find the SoftwareUnit
PlcUnitProvider plcUnitProvider = software.GetService<PlcUnitProvider>
();
plcUnit = plcUnitProvider.UnitGroup.Units.Find(plcUnitName);
if (plcUnit != null)
{
//Do something
UnitAccessType att = readBlockAttribute(plcUnit);
IList<object> atts = readBlockAttributeList(plcUnit);
```

```
}
}
return plcUnit;
}
```

See also

Opening a project

1.2 Working with software unit

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

The units are important parts of the PLC's Programming and can be found on the PlcSoftware.

The unit provider is retrieved from PlcSoftware via GetService and the PlcUnitComposition can be retrieved from it.

You can perform the following elementary operations for units while using TIA Portal Openness:

- Creating units
- Deleting units
- Renaming units

Program code: Creating units

Modify the following program code to create units.

```
Copies the following program code to clipboard.
private static void CreateUnit(PlcUnitProvider provider,
string plcUnitName)
{
PlcUnitComposition unitComposition =
provider.UnitGroup.Units;
//Creating Unit
PlcUnit unit1 = unitComposition.Create(plcUnitName);
```

```
}
```

Error will occur if you want to create a unit with which a name already exists. A Recoverable Exception will be thrown with the following message: "Unit with name'Unit1' already exists.

```
Copies the following program code to clipboard.
PlcUnit unit1 = unitComposition.Create("Unit1");
...
...
try
{
PlcUnit unit2 = unitComposition.Create("Unit1");
}
catch (EngineeringTargetInvocationException  e)
{
Console.WriteLine(e.Message);
}
```

Exception handling: Creating units

Error will occur if you provide the wrong name, for example starting with whitespace or containing invalid character or name is too long. A Recoverable Exception will be thrown with the following error message: "The value of attribute 'Name' contains an invalid character at Position 0."

```
Copies the following program code to clipboard.
private static void CreateUnit(PlcUnitProvider provider,
string plcUnitName)
{
PlcUnitComposition unitComposition =
provider.UnitGroup.Units;
//Creating Unit
try
{
PlcUnit unit1 = unitComposition.Create(plcUnitName);
}
```

```
catch (EngineeringTargetInvocationException e)
{
Console.WriteLine(e.Message);
}
}
```

Error will occur if you want to create a unit with which a name already exists. A Recoverable Exception will be thrown with the following message: "Unit with name'Unit1' already exists.

```
Copies the following program code to clipboard.
private static void CreateUnit(PlcUnitProvider provider,
string plcUnitName)
{
PlcUnitComposition unitComposition =
provider.UnitGroup.Units;
try
{
PlcUnit unit2 = unitComposition.Create("Unit1");
}
catch (EngineeringTargetInvocationException  e)
{
Console.WriteLine(e.Message);
}
```

Program code: Deleting units

Modify the following program code to delete units.

```
Copies the following program code to clipboard.
private static void DeleteUnit(PlcUnitProvider unitProvider,
string plcUnitName)
{
unitProvider.UnitGroup.Units.Find(plcUnitName).Delete();
}
```

You will encounter NullReferenceException in case of the Find( ) method from above example code does not provide a reference to a Unit.

Program code: Renaming units

You can set the attributes in several ways. It can be adjusted via value assignment or call of SetAttribute as well as SetAttributes. The Name property can be validated by GetAttribute or GetAttributes. To be able to use SetAttribute or GetAttribute the object should be casted to IEngineeringObject.

Modify the following program code to rename units.



```
Copies the following program code to clipboard.
private void RenameUnit(PlcUnitProvider provider)
{
PlcUnitComposition unitComposition =
provider.UnitGroup.Units;
//Set Value
PlcUnit unit1 = unitComposition.Create("Unit1");
unit1.Name = "Unit1_new";
//Using SetAttributes():
var attrList = new List<KeyValuePair<string, object>>()
{
new KeyValuePair<string, object>("Name", "Unit2_new")
};
unit1.SetAttributes(attrList);
//Using SetAttribute()
PlcUnit unit2 = unitComposition.Create("Unit2");
IEngineeringObject unit = (IEngineeringObject)unit2;
unit.SetAttribute("Name", "Unit2_new");
}
```

Error will occur if you provide wrong name, for example starting with whitespace, or containing invalid character. A Recoverable Exception will be thrown with the error message: "The value of attribute 'Name' contains an invalid character at Position 0."



```
Copies the following program code to clipboard.
PlcUnit unit1 = unitComposition.Create("Unit_1");
try
{
```

```
unit1.Name = "Unit_1";
}
catch (EngineeringTargetInvocationException e)
{
Console.WriteLine(e.Message);
}
```

Error will occur if you want to rename a unit with a name which already exists. A recoverable exception will be thrown with the error message: "The property 'Name' has an invalid value: 'Unit1'. A software unit with the same name already exists."

```
 Copies the following program code to clipboard.
PlcUnit unit1 = unitComposition.Create("Unit1");
PlcUnit unit2 = unitComposition.Create("Unit_2");
try
{
PlcUnit unit2 = unitComposition.Create("Unit1");
}
catch (EngineeringTargetInvocationException  e)
{
Console.WriteLine(e.Message);
}
```

See also

Opening a project

1.3 Accessing software unit underlying objects

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

You can use the TIA Portal Openness to access:

- The Program blocks system group, its contained groups and blocks recursively
- The PLC data types system group, its contained groups and data types recursively
- The PLC tags system group, its contained groups and tag tables recursively

Program code: Accessing program block and block groups

Modify the following program code to access the program block under the current PLC unit by retrieving the program block composition and iterating through its contained block:

```
private void AccessProgramBlock(PlcUnit m_PlcUnit)
{
PlcBlockComposition blockComposition =
m_PlcUnit.BlockGroup.Blocks;
foreach (PlcBlock block in blockComposition)
{
//usage of block
}
}
```

Modify the following program code to access the program block group under the current PLC unit by retrieving the program block group composition, and iterating through its contained groups:

```
private void AccessProgramBlockGroup(PlcUnit m_plcUnit)
{
PlcBlockUserGroupComposition usergroupComposition =
m_PlcUnit.BlockGroup.Groups;
foreach (PlcBlockUserGroup group in usergroupComposition)
{
PlcBlockComposition blockComposition = group.Blocks;
foreach (PlcBlock block in blockComposition)
{
//…
//usage of the block
```

```
//…
}
}
}
```

Program code: Accessing PLC data types and data type groups

Modify the following program code to access PLC data types under the current PLC unit by retrieving the PLC data type composition and iterating through its contained data types:



```
Copies the following program code to clipboard.
private void AccessPlcDatatypes(PlcUnit m_PlcUnit)
{
PlcTypeComposition typeComposition =
m_PlcUnit.TypeGroup.Types;
foreach (PlcType type in typeComposition)
{
// …
// usage of the type
// …
}
}
```

Modify the following program code to access PLC data type group under the current PLC unit by retrieving the PLC data type composition and iterating through its contained groups:



```
Copies the following program code to clipboard.
private void AccessPlcDatatypeGroup(PlcUnit m_PlcUnit)
{
PlcTypeUserGroupComposition usergroupComposition =
m_PlcUnit.TypeGroup.Groups;
foreach (PlcTypeUserGroup group in usergroupComposition)
{
PlcTypeCompostion typeComposition = group.Types;
foreach (PlcType type in typeComposition)
{
```

```
// …
// usage of the type
// …
}
}
}
```

Program code: Accessing PLC tag tables and tag table goups

Modify the following program code to access the PLC tag tables under the current PLC unit by retrieving the PLC tag table composition and iterating through its contained tag tables:

```
Copies the following program code to clipboard.
private void AccessPlcTagTables(PlcUnit m_PlcUnit)
{
PlcTagTableComposition tagtableComposition =
m_PlcUnit.TagTableGroup.TagTables;
foreach (PlcTagTable type in tagtableComposition)
{
//…
// usage of the tag table
//…
}
}
```

Modify the following program code to access PLC tag table groups under the current PLC unit by retrieving the PLC tag table group composition and iterating through its contained groups:

```
Copies the following program code to clipboard.
private void AccessPlcTagTableGroups(PlcUnit m_PlcUnit)
{
PlcTagTableUserGroupComposition usergroupComposition =
m_PlcUnit.TagTableGroup.Groups;
foreach (PlcTagTableUserGroup group in usergroupComposition)
{
PlcTagTableComposition tagtableComposition = group.TagTables;
```

```
foreach (PlcTagTable type in tagtableComposition)
{
//…
// usage of the tag table
//…
}
}
}
```

1.4 Accessing to existing relations of a unit

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

You can use the TIA Portal Openness to access all the existing relations of a unit so that you can read relation properties.

The following attributes are supported by the unit relation in the Openness:

Attribute name	Type	description
RelationType	UnitRelationType	Type of the relation
RelatedObject	String	Contains the name of the accessible element

The following ENUM values are provided for the attribute RelationType:

- Software Unit
- Non-Unit DB
- TO DB

Program code: Accessing relations

You can get relations, its relation type and the name of the related object in several ways.

Modify the following program code for getting the relation composition:

```
Copies the following program code to clipboard.
private void AccessRelationComposition(PlcUnitProvider
provider)
{
//assuming existing units
m_PlcUnit = provider.UnitGroup.Units[0];
PlcUnitRelationComposition unitRelations =
m_PlcUnit.Relations;
}
```

Modify the following program code for getting relation by indexer:

```
Copies the following program code to clipboard.
private void AccessRelationByIndex(PlcUnitProvider provider)
{
m_PlcUnit = provider.UnitGroup.Units[0]; //assuming existing
units
PlcUnitRelation unitRelation = m_PlcUnit.Relations[1];
}
```

Modify the following program code for getting relations by iterating through them:

```
Copies the following program code to clipboard.
private void AccessUnitRelations(PlcUnitProvider  provider)
{
//assuming existing units
m_PlcUnit = provider.UnitGroup.Units[0];
PlcUnitRelationComposition unitRelations =
m_PlcUnit.Relations;
foreach (PlcUnitRelation relation in unitRelations)
{
// using 'relation'
}
}
```

Modify the following program code for getting the relation type of a unit by the RelationType attribute:



```
private void AccessUnitRelationType(PlcUnitProvider
provider)
{
//assuming existing units
m_PlcUnit = provider.UnitGroup.Units[0];
UnitRelationType unitRelationType =
m_PlcUnit.Relations[2].RelationType;
}
```

Modify the following program code for getting the name of the related object of a unit by the RelatedObject attribute:



```
private void AccessUnitRelatedObject(PlcUnitProvider
provider)
{
//assuming existing units
m_PlcUnit = provider.UnitGroup.Units[0];
string unitRelatedObjectName =
m_PlcUnit.Relations[1].RelatedObject;
}
```

Modify the following program code to identify relation in the collection using Find with the name of accessible element (RelatedObject):



```
private void
SearchUnitRelationFromRelatedObject(PlcUnitProvider provider)
{
//assuming existing units
m_PlcUnit = provider.UnitGroup.Units[0];
PlcUnitRelation relation =
m_PlcUnit.Relations.Find(unitRelatedObjectName);
```

```
}
```

1.5 Updating software unit properties

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to update unit properties such as Author and Comment. You cannot set and get the comment property directly as it is a MultilingualText. The Comment Item's text property can be updated.

The Comment.Item contains the cultures of the TIA Portal project. It can be adjusted via value assignment. The project languages are indexed in the Comment.Item composition, 0 marked as the first default project language. If there are more languages to be set, you can iterate through it, otherwise if the culture does not exist, an EngineeringTargetInvocationException is thrown.

Program code: Update unit's author property

Modify the following program code to modify unit's author property.



```
Copies the following program code to clipboard.

private static void UpdateUnitProperty(PlcUnit unit1, PlcUnit
unit2, string  newAuthor = "Z012345")
{
//Set value
unit1.Author = newAuthor;
//Using SetAttributes()
var attrList = new List<KeyValuePair<string,  object>>()
{
new KeyValuePair<string, object>("Author", newAuthor)
};
unit1.SetAttributes(attrList));
//Using SetAttribute()
IEngineeringObject unit = (IEngineeringObject)unit2;
```

```
unit.SetAttribute("Author", newAuthor);
}
```

Error Message

Error will occur if you try to use the wrong value, for example starts with space, or contains illegal characters. An Recoverable Exception will be thrown with the following error message: "The value of attribute 'Author' contains an invalid character at Position 0."

```
Copies the following program code to clipboard.
private static void UpdateUnitProperty(PlcUnitProvider provider, string
plcUnitName)
{
PlcUnitComposition unitComposition = provider.UnitGroup.Units;
//Creating Unit
PlcUnit unit1 = unitComposition.Create(plcUnitName);
try
{
unit1.Author = " Author";
}
catch (EngineeringTargetInvocationException e)
{
Console.WriteLine(e.Message);
}
```

Program code: Update unit's comment property

You cannot set or get the unit 'Comment' property directly as it is a MultilingualText. You can update the Comment Item's Text Property. The Comment.Item contains the cultures of the TIA Project. It can be adjusted via value assignment. The project languages are indexed in the Comment.Item composition, 0 marked as the first default project language. If there are more languages set, you can iterate on it, otherwise if the culture does not exist, an EngineeringTargetInvocationException is thrown.

```
Copies the following program code to clipboard.
private static void UpdateUnitProperty(PlcUnit unit1)
{
//Setting the default first culture comment Item text:
```

```
unit1.Comment.Items[0].Text = "new Comment";
//Setting other culture comment Item Text:
unit1.Comment.Items[1].Text = "neuro Kommentar";
}
```

Error Message

Error will occur if you try to use the SetAttribute, SetAttributes, GetAttribute, GetAttributes. An EngineeringNotSupportedException will be thrown with the following error message: "Comment is not supported by type 'Siemens.Engineering.SW.Units.PlcUnit'."



```
Copies the following program code to clipboard.
private static void UpdateUnitProperty(PlcUnit unit1)
{
try
{
  ((IEngineeringObject)unit1).SetAttribute("Comment", "new
comments");
}
catch (EngineeringNotSupportedException e)
{
  Console.WriteLine(e.Message);
}
}
```

Error will occur if you try to refer a non existing culture. An EngineeringTargetInvocationException will be thrown with the following error message: "The argument 'index' (3) is outside the valid value range."



```
Copies the following program code to clipboard.
private static void UpdateUnitProperty(PlcUnit unit1)
{
try
{
 unit1.Comment.Items[3].Text = "new Comment";
}
catch (EngineeringTargetInvocationException e)
```

```
{
 Console.WriteLine(e.Message)
}
}
```

1.6 Publishing software unit object

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness .
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

You can use the TIA Portal Openness to get and set the Access attribute of the underlying objects of units so that you can control their accessibilty between units.

The following objects are supported in Openness units:

- Program blocks and data blocks - except OBs
- PLC types

The following possible value of the Access attribute represented as enumeration UnitAccessType:

- Published
- Unpublished

Note

The attribute 'Access' is only available on objects located under a unit, so if it is going to be set or get not under PlcUnit related objects a EnginneringNotSupportedException is thrown. In case of a wrong value (e.g.invalid enum values) EngineeringTargetInvocationException is thrown.

Program code: Configuring the Access attribute of PLC blocks

```
Copies the following program code to clipboard.

private void AccessAttributeOfPLCBlocks(PlcUnit plcUnit,
string blockName = "FB_1")
{
```

```
//Getting attribute value with GetAttribute
...
PlcBlock block = plcUnit.BlockGroup.Find(blockName);
UnitAccessType currentAccessValue =
(UnitAccessType)block.GetAttribute("Access");
...
//Getting attribute value with GetAttributes
...
PlcBlock block = plcUnit.BlockGroup.Block.Find(blockName);
List<string> attrList = new List<string>();
{
"Access"
};
IList<object> getAttributeValues =
block.GetAttributes(attrList));
…
//Setting attribute value with SetAttribute
…
PlcBlock block = plcUnit.BlockGroup.Blocks.Find(blockName);
UnitAccessType newAccessValue = UnitAccessType.Published;
block.SetAttribute("Access", newAccessValue);
…
//Setting attribute value with SetAttributes
…
PlcBlock block = plcUnit.BlockGroup.Blocks.Find(blockName);
UnitAccessType newAccessValue = UnitAccessType.Published;
IList attrList = new List<KeyValuePair<string, object>>()
{
new KeyValuePair<string, object>("Access", newAccessValue),
};
block.SetAttributes(attrList);
…
```

```
}
```

Program code: Configuring the Access attribute of PLC types

```
Copies the following program code to clipboard.
```

```
private void AccessAttributeOfPLCTypes(PlcUnit plcUnit,
string typeName = "UDT_1")
{
//Getting attribute value with GetAttribute
PlcType type = PlcUnit.TypeGroup.Types.Find(typeName);
UnitAccessType currentAccessValue =
(UnitAccessType)type.GetAttribute("Access");
//Getting attribute value with GetAttributes
PlcType type = m_PlcUnit.TypeGroup.Types.Find(typeName);
List<string> attrList = new  List<string>()
{
"Access"
};
IList<object> getAttributeValues =
type.GetAttributes(attrList));
//Setting attribute value with SetAttribute
PlcType type = m_PlcUnit.TypeGroup.Types.Find(typeName);
UnitAccessType newAccessValue = UnitAccessType.Published;
type.SetAttribute("Access", newAccessValue);
//Setting attribute value with SetAttributes
PlcType type = m_PlcUnit.TypeGroup.Types.Find(typeName);
UnitAccessType newAccessValue = UnitAccessType.Published;
IList attrList = new List<KeyValuePair<string, object>>()
{
new KeyValuePair<string, object>("Access", newAccessValue),
};
type.SetAttributes(attrList));
}
```

See also

Connecting to the TIA Portal
Opening a project

1.7 Adding external sources in units

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to add an external source under a software units. You can add the source files such as .SCL,. DB, UDT under units.

You can perform the following tasks with external source group under software units while using Openness API :

- Adding ExternalSourceGroup under units
- Generating blocks/UDTs from an external source under software units
- Exporting source from blocks/UDTs under units

Program code: Adding an external source group

Modify the following program code to add an external source under a software unit:



```
Copies the following program code to clipboard.

private void AddExternalSourceGroup(PlcUnitProvider unitProvider, string
plcUnitName = "Unit1")

{

PlcUnit newUnit = unitProvider.UnitGroup.Units.Create(plcUnitName);

PlcExternalSource unitExternalSource =
newUnit.ExternalSourceGroup.ExternalSources.CreateFromFile(externalSourc
GetFilePath(externalSourceFilename));

}
```

Program code: Generating blocks/UDT from source

Modify the following program code to generate blocks\UDTs with void return types from an external source under a software unit:



```
Copies the following program code to clipboard.
```

```
private void GenerateBlocksFromSource(PlcExternalSource
unitExternalSource)
{
unitExternalSource.GenerateBlocksFromSource();
}
```

Modify the following program code to generate blocks / UDT with collection of blocks get generated from an external source under a software unit:

```
Copies the following program code to clipboard.
private static IList<IEngineeringObject>
GenerateBlocksFromSource(PlcExternalSource unitExternalSource)
{
// Blocks/UDTs gets generated only if there are no compilation errors
IList<IEngineeringObject> generatedBlocksUnderUnitList =
unitExternalSource.GenerateBlocksFromSource(GenerateBlockOption.None);
// Blocks/UDTs will be generated regardless of any compilation errors
IList<IEngineeringObject> generatedBlocksUnderUnitList =
unitExternalSource.GenerateBlocksFromSource(GenerateBlockOption.KeepOnE
}
```

Program code: Exporting source from blocks / UDTs

Modify the following program code to generate source from blocks under a software unit:

```
Copies the following program code to clipboard.
private void GenerateSourceFromBlocks(PlcUnit newUnit)
{
// For blocks
PlcBlock unitPlcBlock =
newUnit.BlockGroup.Blocks.Find(generatedBlockName);
// Generate source from blocks with dependencies
newUnit.ExternalSourceGroup.GenerateSource(new[]
{unitPlcBlock}, new FileInfo(outputFileGeneratedPath),
GenerateOptions.WithDependencies);
// Generate source from blocks without dependencies
```

```
newUnit.ExternalSourceGroup.GenerateSource(new[]
{unitPlcBlock}, new FileInfo(outputFileGeneratedPath),
GenerateOptions.None);
}
```

Modify the following program code to generate source from UDTs under a software unit:

```
Copies the following program code to clipboard.
private void GenerateSourceFromUDTs(PlcUnit newUnit)
{
// For UDTs
PlcType unitPlcUdt =
newUnit.TypeGroup.Types.Find(generatedUdtName);
//Generate source from UDTs with dependencies
newUnit.ExternalSourceGroup.GenerateSource(new[]{unitPlcUdt},
new FileInfo(outputFileGeneratedPath),
GenerateOptions.WithDependencies);
//Generate source from UDTs without dependencies
newUnit.ExternalSourceGroup.GenerateSource(new[]{unitPlcUdt},
new FileInfo(outputFileGeneratedPath), GenerateOptions.None);
}
```

Program code: Enumerating external source file group

Modify the following program code to enumerate through the external source file group composition under the current unit:

```
Copies the following program code to clipboard.
private void AccessExternalSourceFileGroup(PlcUnitProvider
unitProvider)
PlcUnit newUnit =
unitProvider.UnitGroup.Units.Find(textBoxAddNewUnit.Text);
PlcExternalSourceComposition unitExtSrcComposition =
newUnit.ExternalSourceGroup.ExternalSources;
foreach (PlcExternalSource unitExtSrc in
unitExtSrcComposition)
{
```

```
unitExtSrc.GenerateBlocksFromSource(GenerateBlockOption.None);
}
```

1.8 Units as mastercopies

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project
- A unit is created
  See Working with software unit

Introduction

You can use the TIA Portal Openness to copy units as mastercopies to project library and global library.

You can perform the following possible tasks with unit as mastercopy while using TIA Portal Openness:

- Create unit as mastercopy in project library from software units
- Create unit as mastercopy in global library from software units
- Recreate unit from mastercopy of project library to software units
- Recreate unit from mastercopy of global library to software units

Program code

Modify the following program code to create new unit as mastercopy in project library from software units using MasterCopyComposition:



```
Copies the following program code to clipboard.
private void IMasterCopySource
CreateUnitAsMastercopyInProjectLibrary(PlcUnitProvider
m_UnitProvider, string plcUnitName = "Unit_1")
{
PlcUnitProvider m_UnitProvider =
plc.GetService<PlcUnitProvider>();
PlcUnitComposition m_UnitComposition =
m_UnitProvider.UnitGroup.Units;
```

```
IMasterCopySource m_UnitAsMasterCopy =
(IMasterCopySource)m_SoftwareUnit1;//Assuming that m_SoftwareUnit1 is
present in the PLC
m_ProjectLibrary.MasterCopyFolder.MasterCopies.Create(m_UnitAsMasterCopy
}
```

Modify the following program code to create unit as mastercopy in global library from software units:

```
Copies the following program code to clipboard.
private void IMasterCopySource
CreateUnitAsMasterCopyInGlobalLibrary(PlcUnit m_SoftwareUnit1, TiaPortal
m_TiaPortal)
{
// ...
IMasterCopySource m_UnitAsMasterCopy =
(IMasterCopySource)m_SoftwareUnit1;//Assuming that m_SoftwareUnit1 is
present in the PLC
m_TiaPortal.GlobalLibraries.Open(libfile, OpenMode.ReadWrite);
GlobalLibrary m_GlobalLibrary = m_TiaPortal.GlobalLibraries[0];
m_GlobalLibrary.MasterCopyFolder.MasterCopies.Create(m_UnitAsMasterCopy)
// ...
}
```

Error will occur if you try to create a new unit in read-only global library. A recoverable exception will be thrown with the message "Cannot write to read-only libraries".

```
Copies the following program code to clipboard.
private void IMasterCopySource
CreateUnitAsMasterCopyInGlobalLibrary(PlcUnit
m_SoftwareUnit1, TiaPortal m_TiaPortal)
{
IMasterCopySource m_UnitAsMasterCopy =
(IMasterCopySource)m_SoftwareUnit1;//Assuming that
m_SoftwareUnit1 is present in the PLC
m_TiaPortal.GlobalLibraries.Open(libfile, OpenMode.ReadOnly);
GlobalLibrary m_GlobalLibrary =
m_TiaPortal.GlobalLibraries[0];
```

```
try
{
m_GlobalLibrary.MasterCopyFolder.MasterCopies.Create(m_UnitAsMasterCopy)
}
catch (Exception e)
{
Console.WriteLine(e.Message);
}
}
```

Modify the following program code to recreate a unit from mastercopy of project library to software units in PNV:



```
Copies the following program code to clipboard.
private void RecreateUnitFromProjectLibrary(Project project,
PlcSoftware software, string plcUnitName = "Unit_2")
{
//...
ProjectLibrary m_ProjectLibrary = project.ProjectLibrary;
PlcUnitProvider m_UnitProvider =
software.GetService<PlcUnitProvider>();
PlcUnitComposition m_UnitComposition =
m_UnitProvider.UnitGroup.Units;
// ...
MasterCopy mc_Unit_2 =
m_ProjectLibrary.MasterCopyFolder.MasterCopies.Find(plcUnitName);
m_UnitComposition.CreateFrom(mc_Unit_2);//Recreate a Unit from
Project Library to SoftwareUnits folder
}
```

Modify the following program code to recreate unit from mastercopy of global library to software units in PNV.



```
Copies the following program code to clipboard.
private void RecreateUnitFromGlobalLibrary(TiaPorta
m_TiaPortal, string plcUnitName = "Unit_2")
```

```
{
// ...
GlobalLibrary m_GlobalLibrary =
m_TiaPortal.GlobalLibraries[0];
// ...
mc_Unit_2 =
m_GlobalLibrary.MasterCopyFolder.MasterCopies.Find("Unit_2");
m_UnitComposition.CreateFrom(mc_Unit_2);//Recreate a unit
from Global Library to SoftwareUnits folder
}
```

1.9 Updating existing relations & create/delete relations

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to modify the relations of a PLC units so that you can manage the accessibility of objects existing in other PLC units or outside PLC units.

You can perform the following types of modification using TIA Portal Openness:

- Creating new relations
- Deleting existing relations
- Modifying existing relations

Program code: Creating new relations

You can create a new relations by providing the appropriate relation type and the name of the related object.

```
 Copies the following program code to clipboard.
private void CreateNewRelation(PlcSoftware plcTarget)
{
private PlcUnitRelation m_SoftwareUnitRelation;
private PlcUnitRelation m_NonUnitDBRelation;
```

```
private PlcUnitRelation m_TODBRelation;
//...
PlcUnitProvider plcUnitProvider =
plcTarget.GetService<PlcUnitProvider>();
m_PlcUnit = plcUnitProvider.UnitGroup.Units[0]; //assuming
existing units
}
```

Modify the following program code to create a new relation accessing object in another Plc units:



```
Copies the following program code to clipboard.
private void CreateRelationForAccessObject(PlcUnit m_PlcUnit,
string plcUnitName = "Unit_2")
{
// ...
//assuming Plc unit "Unit_2" is already existing in the Plc
m_SoftwareUnitRelation =
m_PlcUnit.Relations.Create(plcUnitName,
UnitRelationType.SoftwareUnit);
// ...
}
```

Modify the following program code to create a new relation accessing PLC data block outside of Plc units:



```
Copies the following program code to clipboard.
private void CreateRelationForAcccessDataBlock(PlcUnit
m_PlcUnit, string dataBlockName = "Data_block_1")
{
// ...
//assuming Plc data block "Data_block_1" is already existing
in the Plc
m_NonUnitDBRelation =
m_PlcUnit.Relations.Create(dataBlockName,
UnitRelationType.NonUnitDB);
// ...
```

```
}
```

Modify the following program code to create a new relation accessing a Technological object outside of Plc units:



```
Copies the following program code to clipboard.

private void
CreateRelationForAccessTechnologicalObject(PlcUnit m_PlcUnit,
string technologicalObjectName = "SpeedAxis_1")
{
// ...
 //assuming Technological object "SpeedAxis_1" is already
existing in the Plc
m_TODBRelation =
m_PlcUnit.Relations.Create(technologicalObjectName,
UnitRelationType.TODB);
// ...
}
```

Possible Error cases for creating a new relation:

Modify the following program code to create new relation accessing non existing related object:



```
Copies the following program code to clipboard.

private void
CreateRelationForAccesNonExistingRelatedObject(PlcUnit
m_PlcUnit)
{
//...
 //assuming Plc unit "NonExistingUnit" is not existing in the
Plc yet
m_SoftwareUnitRelation =
m_PlcUnit.Relations.Create("NonExistingUnit",
UnitRelationType. SoftwareUnit);
//...
}
```

Note

In the above program code, the relation is created and no exception is thrown.

Modify the following program code to create a new relation by specifying a name for the related object which is not conform with the TIA naming rules:



```
Copies the following program code to clipboard.

private void CreateRelationForRelatedObject(PlcUnit
m_PlcUnit)
{
//...
m_SoftwareUnitRelation = m_PlcUnit.Relations.Create("
Unit_2", UnitRelationType.SoftwareUnit);
//...
}
```

Note

In the above program code, the relation is not created and a recoverable exception is thrown due to leading spaces.

Modify the following program code to create a new relation from unit to itself:



```
Copies the following program code to clipboard.

private void CreateRelationFromUnit(PlcUnit m_PlcUnit)
{
//...
 //assuming m_PlcUnit is assigned to Plc unit "Unit_1"
m_SoftwareUnitRelation = m_PlcUnit.Relations.Create("Unit_1",
UnitRelationType.SoftwareUnit);
//...
}
```

Modify the following program code to create a new relation by specifying an empty string for the name of the related object:



```
Copies the following program code to clipboard.
```

```
private void CreateRelationForRelatedObject(PlcUnit
m_PlcUnit)
{
//...
m_SoftwareUnitRelation =
m_PlcUnit.Relations.Create(string.Empty,
UnitRelationType.SoftwareUnit);
//...
}
```

Note

In all of the above program codes for error scenarios, the relation is not created and a recoverable exception is thrown.

Program code: Delete existing relations

You can create a new relations by providing the appropriate relation type and the name of the related object.



```
Copies the following program code to clipboard.
private void DeleteExistingRelation(PlcUnitProvider
plcUnitProvider, string plcUnitName = "Unit_2", string
dataBlockName = "DB_Global", string technologicalName =
"Axis_TO")
{
private PlcUnitRelation m_SoftwareUnitRelation;
private PlcUnitRelation m_NonUnitDBRelation;
private PlcUnitRelation m_TODBRelation;
// ...
m_PlcUnit = plcUnitProvider.UnitGroup.Units[0]; //assuming
existing units
m_PlcUnit2 = plcUnitProvider.UnitGroup.Units[1]; //assuming
existing units
m_SoftwareUnitRelation = m_PlcUnit.Relations.Create("Unit_2",
UnitRelationType.SoftwareUnit);
```

```
m_NonUnitDBRelation = m_PlcUnit.Relations.Create("DB_Global",
UnitRelationType.NonUnitDB);
m_TODBRelation = m_PlcUnit.Relations.Create("Axis_TO",
UnitRelationType.TODB);
}
```

You can delete the relations in several ways.

Modify the following program code to delete the relation accessed by indexer:

```
Copies the following program code to clipboard.
private void DeleteUnitRelationByIndex(PlcUnit m_PlcUnit)
{
//...
m_PlcUnit.Relations[0].Delete();
//...
}
```

Modify the following program code to delete the relation directly:

```
Copies the following program code to clipboard.
private void DeleteUnitRelation(PlcUnitRelation
m_SoftwareUnitRelation)
{
// ...
m_SoftwareUnitRelation.Delete();
// ...
}
```

Program code: Modify existing relations

You can modify the relations in several ways:

You can create a new relations by providing the appropriate relation type and the name of the related object.

```
Copies the following program code to clipboard.
private void ModifyingUnitRelation(PlcUnitProvider
plcUnitProvider, string plcUnitName = "Unit_2")
```

```
{
private PlcUnitRelation  m_Relation;
m_PlcUnit = plcUnitProvider.UnitGroup.Units[0];
 //assuming existing units
m_PlcUnit2 = plcUnitProvider.UnitGroup.Units[1];
 //assuming existing units
m_Relation = m_PlcUnit.Relations.Create("Unit_2",
UnitRelationType.SoftwareUnit);
// ...
}
```

Modify the following program code to update the name of the related object by assigning a new value for the RelatedObject attribute directly:

```
Copies the following program code to clipboard.
private void ModifyingUnitRelation(PlcUnitRelation
m_Relation)
{
// ...
m_Relation.RelatedObject = "Unit_3";
// ...
}
```

Modify the following program code to update the name of the related object by assigning a new value for the RelatedObject attribute through SetAttribute:

```
Copies the following program code to clipboard.
private void ModifyingUnitRelation(PlcUnitRelation
m_Relation)
{
//...
m_Relation.SetAttribute("RelatedObject", "Unit_4");
//...
}
```

Modify the following program code to update the name of the related object by assigning a new value for the RelatedObject attribute through SetAttributes:

```
Copies the following program code to clipboard.
private void ModifyingUnitRelation(PlcUnitRelation
m_Relation)
{
// ...
IList attrList = new List<KeyValuePair<string, object>>();
{
new KeyValuePair<string, object("RelatedObject", "Unit_4")
};
m_Relation.SetAttributes(attrList);
// ...
}
```

Possible error cases for modifying existing relations

Modify the following program code to update a relation to access a non existing related object:

```
Copies the following program code to clipboard.
private void ModifyingUnitRelation(PlcUnitRelation
m_Relation)
{
//...
//assuming Plc unit "NonExistingUnit" is not existing in the
Plc yet
m_Relation.RelatedObject = "NonExistingUnit";
//...
}
```

Note

In the above program code, the relation is modified and no exception is thrown.

Modify the following program code to update a relation by specifying a name for the related object which is not confirm with the TIA naming rules:

```
Copies the following program code to clipboard.
```

```
private void ModifyingUnitRelation(PlcUnitRelation
m_Relation)
{
//...
m_Relation.RelatedObject = " Unit_2";
//...
}
```

Modify the following program code to update a relation to access itself:

```
Copies the following program code to clipboard.
private void ModifyingUnitRelation(PlcUnitRelation
m_Relation)
{
//...
//assuming m_Relation is defined under Plc unit "Unit_1"
m_Relation.RelatedObject = "Unit_1";
//...
}
```

Note

In the above program code, the relation is not modified and a recoverable exception is thrown.

Modify the following program code to update a relation in a way which would be a duplicate of another already existing with the same relation type and related object:

```
Copies the following program code to clipboard.
private void ModifyingUnitRelation(PlcUnitRelation
m_Relation)
{
//...
 //assuming a relation with the same relation type and
related object is already existing
m_Relation.RelatedObject = "Unit_2";
//...
```

```
}
```

Modify the following program code to update a relation by specifying an empty string for the name of the related object:

```
Copies the following program code to clipboard.

private void ModifyingUnitRelation(PlcUnitRelation
m_Relation)
{
//...
m_Relation.RelatedObject = string.Empty;
//...
}
```

Note

In all of the above program codes for errors scenarios, the relation is not modified and a recoverable exception is thrown.

1.10 Accessing name value type document

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Introduction

You can use the TIA Portal Openness application to access the name value type documents.

The Siemens.Engineering.SW.PlcDocumentComposition class includes the Find() method, which returns a PlcDocument by name and the Siemens.Engineering.SW.PlcDocument type has a read-only Name property, both introduced in TIA Portal Version 20.

Program code

To access the Named_Value_Type_1.nvt modify the following program code:

```
Copies the following program code to clipboard.
```

```
private void AccessNameValueTypeDocument(PlcSoftware
plcSoftware)
{
    var unitProvider =
plcSoftware.GetService<PlcUnitProvider>();
    var unit = unitProvider.UnitGroup.Units.Find("Unit_1");
    PlcDocumentComposition documentComposition =
unit.TypeGroup.Documents;
    PlcDocument document_NVT2 = documentComposition[0];


}
```

To access the Named_Value_Type_2.nvt modify the following program code:

```
 Copies the following program code to clipboard.
private void AccessNameValueTypeDocument(PlcSoftware
plcSoftware)
{
    var unitProvider =
plcSoftware.GetService<PlcUnitProvider>();
    var unit =
unitProvider.UnitGroup.Units.Find("SafetyUnit");
    PlcDocumentComposition documentComposition =
unit.TypeGroup.Documents;
    PlcDocument document_NVT1 = documentComposition[0];
}
```

1.11 Accessing namespaces for software units

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

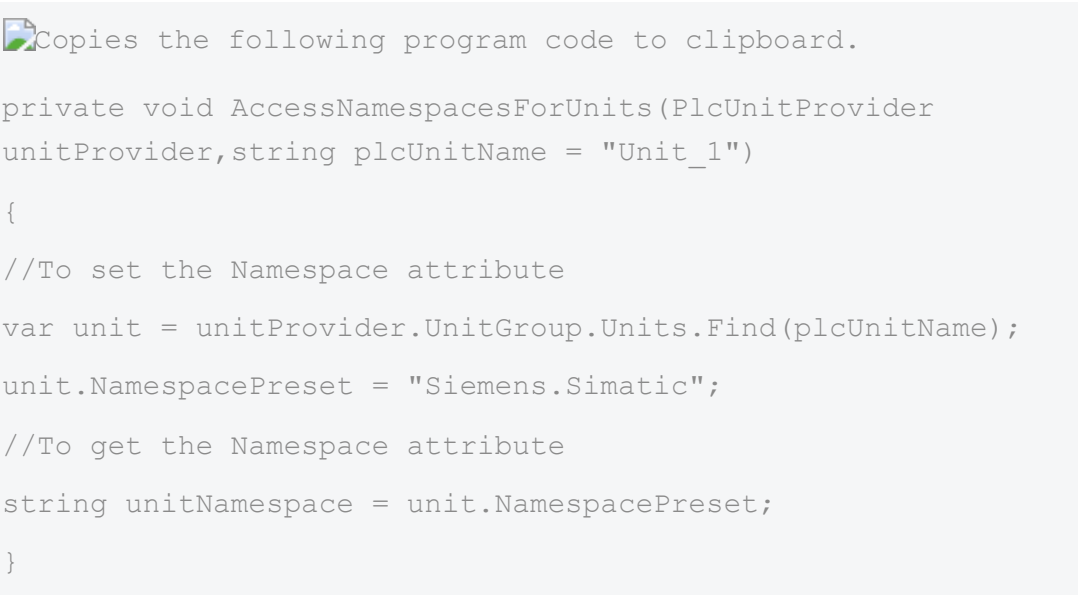
Application

You can use the TIA Portal Openness to set and get the namespace attribute for software unit.

Property Name	Data Type	Access
NamespacePreset	String	Read/Write

Program code

Modify the following program code to set and get the Namespace attribute for unit:



See also

Opening a project

1.12 Accessing namespaces for program blocks

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

You can use the TIA Portal Openness to assign the namespaces attribute for program block under software units via attribute set. The namespace assigned to the program blocks can be exported and imported under units via SimaticML.

Property Name	Data Type	Access
Namespace	String	Read/Write

Program code

Modify the following program code to set and get the Namespace attribute:



```
Copies the following program code to clipboard.
private void static AccessNamespaceForProgramBlock()
var nameSpace = unitBlock.GetAttribute(name:"Namespace");
unitBlock.SetAttribute(name:"Namespace",
value:"Unit.Siemens");
```

Namespace attribute in SimaticML



```
Copies the following program code to clipboard.
</Sections></Interface>
<IsIECCheckEnabled>false</IsIECCheckEnabled>
<Name>Block_1</Name>
<Namespace>Siemens.Simatic</Namespace>
<Number>1</Number>
<ProgrammingLanguage>LAD</ProgrammingLanguage>
<SetENOAutomatically>false</SetENOAutomatically>
<UDABlockProperties/>
<UDAEnableTagReadback>false</UDAEnableTagReadback>
</AttributeList>
<ObjectList>
<MultilingualText ID="1" CompositionName="Comment">
<ObjectList>
<MultilingualTextitem ID="2" CompositionName=":Items">
<AttributeList>
<Culture>en-US</Culture>
<Text/>
```

```
</AttributeList>
</MultilingualTextItem>
</ObjectList>
```

See also

Connecting to the TIA Portal
Opening a project

1.13 Accessing namespaces for UDT

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to set and get the namespaces to the UDT via GetAttribute and SetAttribute under software units. The namespace assigned to the UDTs can be exported and imported under units via SimaticML.

Property name	Data Type	Access
Namespace	String	Read/Write

Program code

Modify the following program code to set and get the Namespace attribute:

```
Copies the following program code to clipboard.
private void AccessNamespaceForUDT()
{
//Namespace can be set and get to the UDTs via the property
unitUDT.Namespace = "Motor";
var namespaceValue = unitUDT.Namespace;
//Namespace can be set and get to the UDTs via the attribute
set
```

```
var namespaceValue = unitUDT.GetAttribute("Namespace");
}
```

Namespace attribute in SimaticML

```
<?xml version="1.0" encoding="utf-8"?>
<Document>
<Engineering version="V18" />
<DocumentInfo>
<Created>2022-08-03T05:28:40.722228Z</Created>
<ExportSetting>None</ExportSetting>
<InstalledProducts>
<Product>
<DisplayName>Totally Integrated Automation
Portal</DisplayName>
<DisplayVersion>V18</DisplayVersion>
</Product>
<OptionPackage>
<DisplayName>TIA Portal Version Control
Interface</DisplayName>
<DisplayVersion>V18</DisplayVersion>
</OptionPackage>
<Product>
<DisplayName>Feature Cycle 2 TIA Portal</DisplayName>
<DisplayVersion>V18</DisplayVersion></Product>
<Product>
<DisplayName>Feature Cycle 5 TIA Portal</DisplayName>
<DisplayVersion>V18</DisplayVersion>
</Product>
<Product>
<DisplayName>STEP 7 Professional</DisplayName>
```

```
<OptionPackage>
<DisplayName>STEP 7 Safety</DisplayName>
<DisplayVersion>V18</DisplayVersion>
</OptionPackage>
</InstalledProducts>
</DocumentInfo>
<SW.Types.PlcStruct ID="0">
<AttributeList>
<Interface><Sections
xmlns="http://www.siemens.com/automation/Openness/SW/Interface/v5">
<Section Name="None">
<Member Name="Element_1" Datatype="Bool" />
</Section>
</Sections></Interface>
<Name>UDT_1</Name>
<Namespace>Siemens</Namespace>
</AttributeList>
<ObjectList>
<MultilingualText ID="1" CompositionName="Comment">
<ObjectList>
<MultilingualTextItem ID="2" CompositionName="Items">
<AttributeList>
<Culture>en-US</Culture>
<Text />
</AttributeList>
</MultilingualTextItem>
</ObjectList>
</MultilingualText>
<MultilingualText ID="3" CompositionName="Title">
<ObjectList>
<MultilingualTextItem ID="4" CompositionName="Items">
```

```
<AttributeList>
<Culture>en-US</Culture>
<Text />
</AttributeList>
</MultilingualTextItem>
</ObjectList>
</MultilingualText>
</ObjectList>
</SW.Types.PlcStruct>
</Document>
```

See also

Connecting to the TIA Portal
Opening a project

1.14 Generating loadable file for software units

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- The TIA Portal project is open
  SeeOpening a project
- The PLC must be compile clean

Introduction

You can use the TIA Portal Openness to generate a loadable file for software units via the following method: `GenerateLoadable(FileInfo path, IEnumerable<PlcUnit> unit, TargetOption targetOption)`

Note

The generation of a loadable file for software units is supported for PLC1518 or its subtype whose firmware is V2.8 or higher.

Restriction

Loadable files can only be created for such software units, that

- do not contain any system blocks
- do not contain any ProDiag blocks
- do not contain any failsafe blocks
- do not contain OBs of any other type than ProgramCycle
- do not contain block with dynamic binding
- do not contain any technological objects
- do not contain any user datatypes

Parameter and Enum

The GenerateLoadable() accepts the following parameters to generate a loadable file for software units:

Parameter name	Data type	Descr
path	FileInfo	Specif the na and th storag locati for th loada file
units	System.Collections.Generic.IEnumerable<Siemens.Engineering.SW.Units.PlcUnit>	Specif the softw units are to includ the loada file
targetOption	Siemens.Engineering.SW.Loader.TargetOption	Specif wheth the loada file is create a real or for

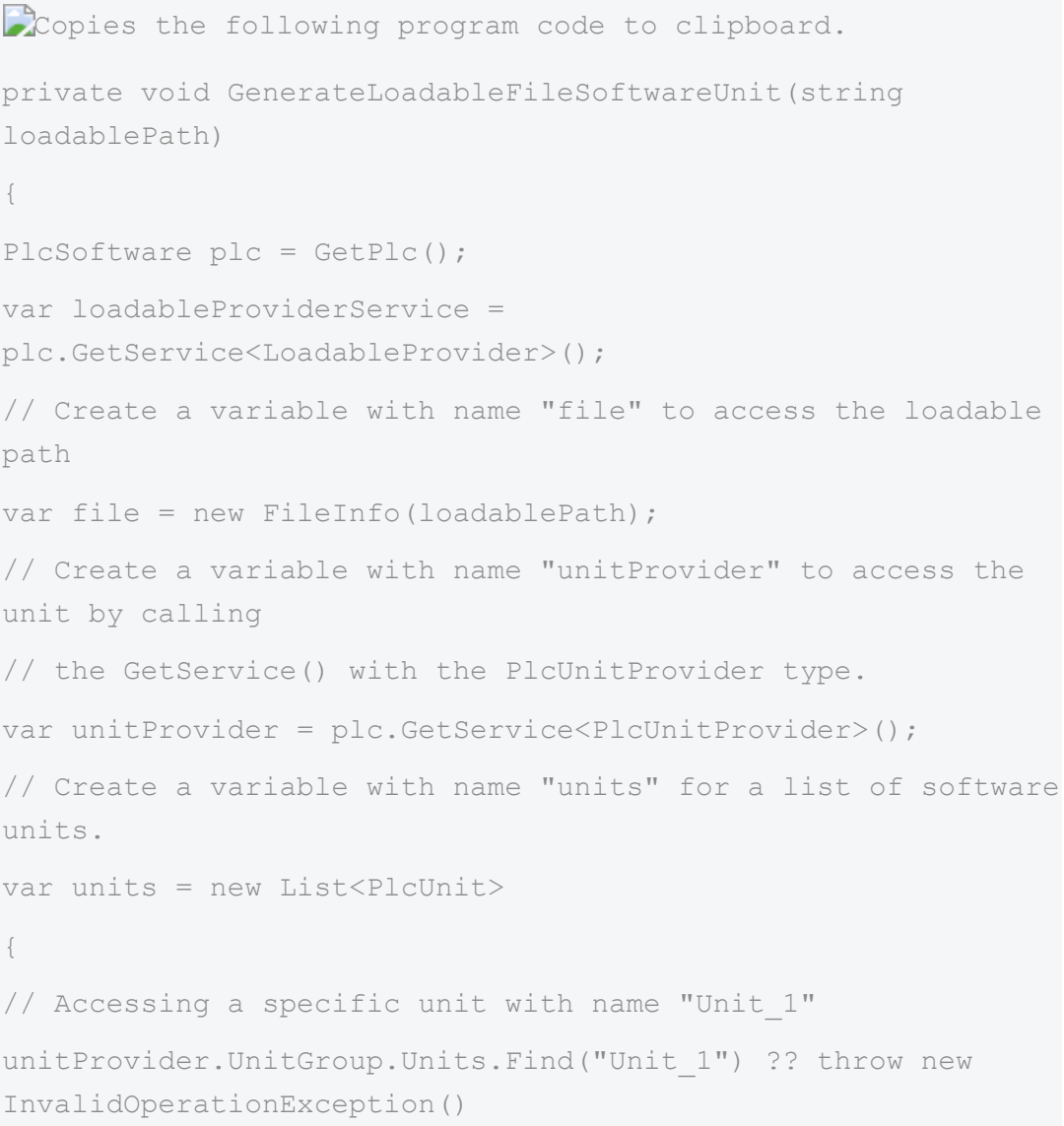
		simulated PLC	If TargetOption.PlcSim is passed, all blocks must have been compiled with simulation support

The TargetOption parameter uses the following enum values to generate a loadable file:

Value	Description
TargetOption.None	The default value, should not be used.
TargetOption.Plc	To generate a loadable file for a real PLC.
TargetOption.PlcSim	To generate a loadable file for a simulated PLC.

Program code

To generate a loadable file for software units modify the following program code:



```
};
// The loadableProviderService generates a loadable file for
units by calling
// the GenerateLoadable() with respective parameters.
try
{
loadableProviderService.GenerateLoadable(file, units,
TargetOption.PlcSim);
}
// An EngineeringTargetInvocation exception is thrown
// if generation of the loadable file fails.
// The exception message contains hints on
// what caused the loadable file creation to fail.
catch (EngineeringTargetInvocationException e)
{
Console.WriteLine("Generating the loadable file failed.");
Console.WriteLine(e);
}
}
```

Exception

The following possible exception message are listed below:

- It is only possible to create a loadable file from a whole software unit, not from a single block inside of a unit.
- The selected blocks are from different PLCs.
- Destination directory not found: '{0}'.
- It is not possible to create a loadable file from failsafe block: '{0}'.
- Destination file already exists: '{0}'.
- The loadable file is not accessible.
- An internal error occured.
- The PLC has to be compiled.
- Internal serialization of loadable file failed.
- Internal serialization failed, because loadable file was disposed.

- The specified path, file name, or both exceed the system-defined maximum length.
- The PLC '{0}' is not supported.
- The parent PLC of the service and of the target are different.
- Some block(s) cannot be simulated. If a block is a library block, use a library with simulation support. Otherwise, select the option "Support simulation during block compilation" in the project properties and recompile the block.
- It is not possible to create a loadable file from a system block: '{0}'.
- Access to file is denied.
- Software units containing PLC data types are not supported.
- It is not possible to create a loadable file from selected object(s).
- It is not possible to create a loadable file from OB '{0}', because it does not have the event class 'Program cycle'.
- It is not possible to create a loadable file from a Prodiag block: '{0}'.
- It'is not possible to create a loadable file from selected object: '{0}'.
- It Is not possible to create a loadable file from block '{0}' with dynamic binding enabled.

See also

Generating loadable file for blocks outside of software units

Functions for fail-safe unit

2

2.1 Generating fail-safe unit

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to add the fail-safe program into software units. The Safety-Software-Unit is a system generated, one-and-only SW-Unit which can be hosted by Fail-safe PLCs only. For information on software unit, see Working with software unit

You can generate the fail-safe program under software unit only if the SafetyInUnit option is enabled via ManagementOfFail-safeInSoftwareUnitsEnvironment API. You can create a new Fail-safe PLC which supports software units. A SafetyUnit is then created automatically which serves as the one and only host of the F-program in software unit environment.

Method

The TIA Portal Openness provides the following methods which can be used to generate failsafe-PLC with fail-safe program:

Method	Descriptiom
bool GenerationOfDefaultFailsafeProgram()	To retrieve the setting for generating the default fail-safe program.
void GenerationOfDefaultFailsafeProgram(bool)	To enable/disable the the generation of the default fails-safe program when creating a new fail-safe PLC.

The TIA Portal Openness provides the following methods which can be used to genearte failsafe program under SafetyUnit:

Method	Descriptiom

bool ManagementOfFailsafeInSoftwareUnitsEnvironment()	To retrieve the state of the respective setting.
void ManagementOfFailsafeInSoftwareUnitsEnvironment (bool)	To enable/disable the hosting of the fail-safe program within a Safety Unit.

Program code



```
Copies the following program code to clipboard.
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
using System.Security;
```

```
using Siemens.Engineering.SW.Units;

namespace GenerateFailSafeUnit

{

internal class Program

{

private static void Main(string[] args)

{

// Generate default fail-safe program under PLC:

GlobalSettings globalSettingsService =
myTiaPortal.GetService<GlobalSettings>();

// Get the value

bool isDefaultFailsafeProgram =
globalSettingsService.GenerationOfDefaultFailsafeProgram();

// Set the value

globalSettingsService.GenerationOfDefaultFailsafeProgram(true);

// Generate fail-safe program under SafetyUnit environment

GlobalSettings globalSettingsService =
myTiaPortal.GetService<GlobalSettings>();

// Get the value

bool isSafetyUnitSupported =
globalSettingsService.ManagementOfFailsafeInSoftwareUnitsEnvironment();

// Set the value

globalSettingsService.ManagementOfFailsafeInSoftwareUnitsEnvironment(tru

}

}

}
```

See also

Opening a project

2.2 Accessing the SafetyUnit

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to access safety unit using the type "PlcSafetyUnitComposition" UnitGroup. Once the PLC has been created, the Safety Unit can neither be created nor deleted.

The Safety Unit is a one-and-only system generated Unit.

Attribute

The TIA Openness supports the following property to access the safety unit created under the SoftwareUnits folder in TIA Portal:

Property name	Datatype	Access
SafetyUnits	PlcSafetyUnitComposition	Read/Write

Program code

Modify the following program code to access the fail-safe unit:



```
Copies the following program code to clipboard.
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
```

```
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
using System.Security;
using Siemens.Engineering.SW.Units;
namespace AccessingSafetyUnit
{
internal class Program
{
private static void Main(string[] args)
{
var plcUnitProvider = plcSoftware.GetService<PlcUnitProvider>
();
PlcSafetyUnit safetyUnit =
plcUnitProvider.UnitGroup.SafetyUnits.Find("SafetyUnit");
PlcSafetyUnitComposition plcSafetyUnit =
plcUnitProvider.UnitGroup.SafetyUnits; //- Create method is
not accessible like
//in standard unit and compile error is thrown
}
}
}
```

See also

Opening a project

2.3 Creating/deleting SafetyUnit relations

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

You can use the TIA Portal Openness to create and delete the safety unit relation. The TOs and GlobalDBs can be ceated and deleted in the same way as standard unit relations.

Program code

Modify the following program code to create and delete the relations:



```
Copies the following program code to clipboard.
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
```

```
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
using System.Security;
using Siemens.Engineering.SW.Units;
namespace CreateDeleteRelationshipsForTheSafetyUnit
{
internal class Program
{
private static void Main(string[] args)
{
// Create relation
PlcSoftware plcSoftware = ...;
var plcUnitProvider = plcSoftware.GetService<PlcUnitProvider>
();
var safetyUnit =
plcUnitProvider.UnitGroup.SafetyUnits.Find("SafetyUnit");
var unitRelation = safetyUnit.Relations.Create("Unit_1",
UnitRelationType.SoftwareUnit);
// Delete relation
unitRelation.Delete();
}
}
}
```

See also

Opening a project

2.4 Creating/Accessing SafetyUnit elements/objects

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal

- A project is open
  See Opening a project

Application

You can use the TIA Portal Openness to create, delete, access, update, and publish the underlying objects such as blocks, tags, UDT of safety unit. You can work with objects under safety unit likewise in the standard unit except for fail-safe system generated blocks. You can also export and import the blocks under safety unit using TIA Portal Openness.

Program code

Modify the following program code to create and access safety unit tags:



```
PlcUnitProvider plcUnitProvider =
sw.GetService<PlcUnitProvider>();
PlcSafetyUnit safetyUnit =
plcUnitProvider.UnitGroup.SafetyUnits.Find("SafetyUnit");
// Create tagtable
var tagTable =
safetyUnit.TagTableGroup.TagTables.Create("Tag_Table_1");
// Create tag
var newTag = tagTable.Tags.Create("tag_1");
// Delete tag
newTag.Delete();
// Search tag
var expectedTag = tagTable.Tags.Find("tag_1");
```

Modify the following program code to export and import safety unit blocks:



```
PlcSafetyUnit safetyUnit =
plcUnitProvider.UnitGroup.SafetyUnits.Find("SafetyUnit");
var unitBlock = safetyUnit.BlockGroup.Blocks.Find("Block_2");
// Export the block
unitBlock.Export(new
FileInfo(@"D:\Export\SafetyBlock_unit.xml"),
ExportOptions.None);
```

```
// Import of block
safetyUnit.BlockGroup.Blocks.Import(new FileInfo(
@"C:\Users\z003hzed\Desktop\Block1.xml"),
ImportOptions.Override);
```

Program code

Modify the following program code:

 Copies the following program code to clipboard.

```
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
using System.Security;
```

```
using Siemens.Engineering.SW.Units;

namespace CreatingAndAccessingSafetyUnitElement

{

internal class Program

{

private static void Main(string[] args)

{

// To create and access safety unit tags

PlcUnitProvider plcUnitProvider =
sw.GetService<PlcUnitProvider>();

PlcSafetyUnit safetyUnit =
plcUnitProvider.UnitGroup.SafetyUnits.Find("SafetyUnit");

// Create tagtable

var tagTable =
safetyUnit.TagTableGroup.TagTables.Create("Tag_Table_1");

// Create tag

var newTag = tagTable.Tags.Create("tag_1");

// Delete tag

newTag.Delete();

// Search tag

var expectedTag = tagTable.Tags.Find("tag_1");

// To export and import safety unit blocks

PlcSafetyUnit safetyUnit =
plcUnitProvider.UnitGroup.SafetyUnits.Find("SafetyUnit");

var unitBlock = safetyUnit.BlockGroup.Blocks.Find("Block_2");

// Export the block

unitBlock.Export(new
FileInfo(@"D:\Export\SafetyBlock_unit.xml"),
ExportOptions.None);

// Import of block

safetyUnit.BlockGroup.Blocks.Import(new FileInfo(
@"C:\Users\z003hzed\Desktop\Block1.xml"),
ImportOptions.Override);
```

```
}
}
}
```

See also

Opening a project

2.5 Exporting/Importing supervisions under SafetyUnit

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to export and import supervision under safety units by accessing supervision provider from safety unit prodiag blocks.

The following restrictions are applied on SafetyUnit:

- Creation, deletion, and copy of the stand-alone SafetyUnit is not allowed via TIA Portal Openness
- Download and upload of SafetyUnit is not allowed via TIA Portal Openness
- Copy of the hosted SafetyUnit into mastercopy of project/global library is not allowed in TIA Portal Openness

Program code

Modify the following program code to export and import supervisions under the safetyunit:

```
Copies the following program code to clipboard.
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
```

```
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
using System.Security;
using Siemens.Engineering.SW.Units;
namespace ExportingImportingSupervisionsUnderSafetyUnit
{
internal class Program
{
private static void Main(string[] args)
{
string filePath = @"E:\Temp\...\fileName.xlsx";
PlcSafetyUnit safetyUnit = ...;
// Export from prodiag supervision provider
var proDiagBlock =
safetyUnit.BlockGroup.Blocks.CreateFB("Block_1", true, 1,
ProgrammingLanguage.ProDiag);
var proDiagSupervisionProvider =
proDiagBlock.GetService<SupervisionProvider>();
```

```
var superVisionResult =
proDiagSupervisionProvider.ExportSupervisionsToXlsx(new
FileInfo(filePath));

// Import from prodiag supervision provider

ImportOptions importOptions = ImportOptions.None;

var superVisionResult =
proDiagSupervisionProvider.ImportSupervisionsFromXlsx(new
FileInfo(filePath), importOptions);

// Export from global supervision provider.

var globalSupervisionProvider =
safetyUnit.GetService<SupervisionProvider>();

var superVisionResult =
globalSupervisionProvider.ExportSupervisionsToXlsx(new
FileInfo(filePath));

// Import from global supervision provider

var superVisionResult =
globalSupervisionProvider.ImportSupervisionsFromXlsx(new
FileInfo(filePath), importOptions);

// Import supervison settings

var superVisionResult =
globalSupervisionProvider.ImportSupervisionSettingsFromXlsx(new
FileInfo(filePath), importOptions);

}

}

}
```

See also

Opening a project

2.6 Publishing blocks under the SafetyUnit

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to publish all the blocks (FC, FBs, DBs, and IDBs) that are created under the safety unit's program blocks folder. All the system generated blocks in the SafetyUnit are excluded from being publishable and a runtime exception is thrown while accesing.

Program code

Modify the following program code to set the published attribute under safetyunit:

 Copies the following program code to clipboard.

```
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
```

```
using System.Security;
using Siemens.Engineering.SW.Units;
namespace PublishingBlocksUnderSafetyUnit
{
internal class Program
{
private static void Main(string[] args)
{
PlcSafetyUnit safetyUnit =
plcUnitProvider.UnitGroup.SafetyUnits.Find("SafetyUnit");
 var safetyBlock =
safetyUnit.BlockGroup.Blocks.Find("Block_2");
//To set the access
safetyBlock.SetAttribute("Access", UnitAccessType.Published);
//To get the access
var access = safetyBlock.GetAttribute("Access");
}
}
}
```

See also

Opening a project